

Loyalty Program — Rapport de compréhension du projet

Plateforme SaaS multi-tenant de fidélisation (Yowyob)

10 juillet 2026

Table des matières

1	1. Vue d'ensemble	1
1.1	1.1 Stack technique	2
2	2. Architecture backend : hexagonale et réactive	2
2.1	2.1 Domaines métier identifiés	3
2.2	2.2 Le moteur de règles (Rule Engine)	3
2.3	2.3 Multi-tenancy	4
2.4	2.4 Intégration externe « Kernel Core »	4
2.5	2.5 Endpoints REST représentatifs	4
2.6	2.6 Historique des migrations Flyway	5
3	3. Frontend : portail Next.js	6
3.1	3.1 Portail admin (/portal)	6
3.2	3.2 Couche d'accès API (src/lib/api.ts)	6
3.3	3.3 Hooks et composants	6
4	4. État d'avancement et feuille de route	7
5	5. Points forts et observations	7

1 1. Vue d'ensemble

Loyalty Program est une plateforme SaaS multi-tenant de fidélisation développée par Yowyob. Elle permet à des entreprises clientes (tenants) de configurer un programme de fidélité complet pour leurs propres clients (membres) : règles de gain de points, portefeuille électronique (wallet), récompenses, parrainage, codes promo, campagnes marketing temporisées, et paliers de fidélité. La plateforme elle-même est vendue en marque blanche sous forme d'abonnements SaaS aux tenants.

Le projet est composé de deux applications distinctes dans le même dépôt :

- un **backend** Java 21 / Spring Boot 3.4 en architecture hexagonale, entièrement réactif (WebFlux + R2DBC) ;
- un **frontend** Next.js 16 (App Router, TypeScript) servant de portail d'administration pour les tenants, ainsi qu'un portail développeur séparé pour l'intégration technique (clés API, webhooks, sandbox).

L'historique Git montre une construction incrémentale par domaines successifs : fondations et wallet, moteur de règles, intégration bonification externe, récompenses, parrainage, codes promo, campagnes/paliers, abonnements SaaS, puis le frontend — ce qui correspond exactement à la feuille de route documentée dans docs/TODO.MD.

1.1 1.1 Stack technique

Couche	Technologies
Backend	Java 21, Spring Boot 3.4/3.5, WebFlux, Spring Data R2DBC, Spring Security OAuth2 Resource Server
Base de données	PostgreSQL (via R2DBC en runtime, JDBC pour Flyway)
Cache / idempotence	Redis (réactif)
Messagerie événementielle	Kafka (Reactor Kafka)
Identité	Keycloak (OAuth2/JWT), + service externe Kernel Core pour l'authentification admin et la résolution tenant
Migrations	Flyway (V001 → V022)
Mapping	MapStruct + Lombok
Tests	JUnit 5, Testcontainers (Postgres, Kafka, Keycloak), ArchUnit, Reactor Test
Observabilité	Actuator + Micrometer/Prometheus
Documentation API	springdoc-openapi (Swagger UI)
Frontend	Next.js 16 (App Router), TypeScript, shadcn/ui, next-intl, Tailwind

2 2. Architecture backend : hexagonale et réactive

Le backend applique une **architecture hexagonale (ports & adapters)** stricte, vérifiée automatiquement au build par HexagonalArchitectureTest (ArchUnit) : le package domain ne doit jamais dépendre de infrastructure ni de api.

```
com.yowjob.loyalty/
├── api/           Contrôleurs WebFlux, DTO, mappers (adaptateurs entrants)
├── application/   Handlers de cas d'usage (orchestrent domaine + ports)
├── domain/        Logique métier pure, ports (interfaces), modèles
├── infrastructure/ Adaptateurs sortants : R2DBC, Redis, Kafka, sécurité
└── shared/        Transverse : contexte multi-tenant, exceptions, logging
```

Chaque domaine métier suit la même structure interne : model/, port/in/ (cas d'usage), port/out/ (dépôts et intégrations externes), service/ (implémentations domaine), event/, exception/. Côté infrastructure, le même patron se répète pour chaque domaine persisté : port sortant → *RepositoryAdapter → *R2dbcRepository (Spring Data R2DBC) → *Entity.

L'ensemble de la chaîne est non-bloquante (Mono/Flux de bout en bout) : HTTP réactif (WebFlux), accès base de données réactif (R2DBC), Kafka réactif (Reactor Kafka), Redis réactif.

Flux de traitement type : Controller → Handler (application) → cas d'usage (port in, domaine) → port out (repository/cache) → adaptateur infrastructure.

2.1 2.1 Domaines métier identifiés

Domaine	Rôle
loyalty	Cœur du moteur de fidélité : règles, points, paliers, compteurs, événements entrants
wallet	Portefeuille électronique du membre : solde, transactions, crédit/débit, gel, OTP
reward	Catalogue de récompenses, octroi de « grants », rédemption / consommation
campaign	Campagnes marketing programmables (bonus multiplicateur ou fixe, planification)
promo	Codes promo : validation, application, quotas d'usage
referral	Programme de parrainage : liens, conversions, anti-fraude basique
subscription	Abonnement SaaS du tenant lui-même à la plateforme (monétisation)
tenant bonification	Modèle tenant, clés API Intégration à une API externe tierce de bonification (« bonusapi »)
webhook	Endpoints webhook sortants et suivi des livraisons
shared	Primitives communes : Money, TenantId, UserId, pagination, audit, exceptions de base

Deux domaines méritent une attention particulière car leur rôle est moins évident :

- **subscription** ne concerne pas les points ou avantages accordés aux membres finaux, mais la facturation SaaS du **tenant** envers Yowjob (plans, cycle de facturation, essai gratuit, renouvellement). C'est le mécanisme de monétisation de la plateforme elle-même, orthogonal au moteur de fidélité.
- **bonification** est un pont HTTP vers un service externe tiers (BONIFICATION_API_BASE_URL, bonusapi.onrender.com) permettant de créditer/débiter un compte externe lié au membre. Peu couplé au reste du domaine (pas de repository R2DBC propre), il s'appuie sur un WebClient, une authentification par identifiants résolus par tenant, un cache Redis du jeton JWT, et un mode « désactivé » (BonificationDisabledAdapter) en no-op si la fonctionnalité n'est pas activée pour le tenant.

2.2 2.2 Le moteur de règles (Rule Engine)

C'est la pièce centrale du système, pilotée par RuleEngine (domain/loyalty/service/). Le pipeline est **Trigger → Conditions → Effects** :

1. Les règles actives sont triées par priorité décroissante, puis filtrées sur leur fenêtre de validité (date de l'événement).
2. Pour chaque règle, on vérifie que `rule.triggerMatches(event)`.
3. Les conditions (ConditionDefinition) sont évaluées séquentiellement par un ConditionEvaluator correspondant, avec sortie anticipée (*fail-fast*) dès qu'une condition échoue.
4. Si toutes les conditions passent, les effets (EffectDefinition) sont exécutés via un EffectExecutor.

Évaluateurs de conditions (service/evaluator/) : montant cumulé (CumulativeAmountEvaluator), nombre cumulé (CumulativeCountEvaluator), premier événement (FirstEventEvaluator),

solde de points (PointsBalanceEvaluator), comparaison de palier (TierEvaluator), fenêtre temporelle — jours/heures (TimeWindowEvaluator).

Exécuteurs d'effets (service/executor/) : crédit de points, crédit du wallet, octroi de récompense, réinitialisation de compteur, envoi de notification, mise à jour de palier.

La résolution est dynamique (patron *Strategy*, méthode `supports(type)`), ce qui rend le moteur extensible sans le modifier — nouveau type de condition ou d'effet = nouvelle classe, sans toucher au cœur du moteur.

Les règles actives sont mises en cache Redis (RuleCacheAdapter) pour éviter une lecture base à chaque événement entrant, l'endpoint `POST /api/v1/events` étant le point d'entrée qui déclenche l'évaluation.

2.3 2.3 Multi-tenancy

Chaque requête porte un `tenantId` extrait du claim JWT `organization_id`. `TenantContextHolder` propage ce contexte via le **Reactor Context** (`contextWrite/deferContextual`) plutôt qu'un `ThreadLocal` — un choix cohérent avec le modèle non-bloquant de WebFlux, où un même thread peut traiter plusieurs requêtes entrelacées.

Deux `WebFilter` mutuellement exclusifs, activés par profil Spring :

- **TenantResolutionFilter** (profil !dev) : extrait le tenant du JWT, le résout via le cache Redis (TenantCacheAdapter) puis, en repli, via un adaptateur appelant le service externe **Kernel Core** ; rejette (401) si le tenant n'est pas actif.
- **DevTenantResolutionFilter** (profil dev) : tente d'extraire le tenant du JWT si présent, sinon injecte un tenant par défaut configurable (`app.dev.default-tenant-id`) avec statut/plan forcés à actif/PRO — ceci simplifie le développement local sans nécessiter un realm Keycloak complet.

Les chemins publics (`/public/`, `/actuator`, `/swagger-ui`, `/v3/api-docs`, `/webjars/`) sont exclus de cette résolution.

2.4 2.4 Intégration externe « Kernel Core »

Le backend délègue l'authentification admin et la résolution tenant/organisation à un service externe, **Kernel Core** (`KERNEL_CORE_URL`). Chaque appel requiert des en-têtes `X-Client-Id` / `X-API-Key`. Kernel Core ne supporte pas le grant `OAuth2 client_credentials` : l'authentification se fait uniquement par ces en-têtes statiques, sans passer par un endpoint de token.

2.5 2.5 Endpoints REST représentatifs

Domaine	Exemples d'endpoints
wallet	<code>GET/POST /api/v1/members/{id}/wallet</code> , <code>/credit</code> , <code>/debit</code> , <code>/confirm-otp</code> , <code>/freeze</code> , <code>/unfreeze</code> , <code>/transactions</code>
loyalty	<code>POST /api/v1/events</code> , <code>POST/GET</code> <code>/api/v1/admin/rules</code> , <code>PATCH</code> <code>.../{ruleId}/activate</code> , <code>GET</code> <code>/api/v1/members/{id}/points</code> , <code>/tier</code>

Domaine	Exemples d'endpoints
reward	/api/v1/admin/rewards (CRUD + activate/pause/archive), /api/v1/members/me/redeem, /grants, /api/v1/reward-grants/{id}/consume
promo	/api/v1/promo/admin/campaigns, /validate, /apply
referral	/api/v1/referral/program, /enroll, /me/convert, /my-referrals
campaign	/api/v1/campaigns (CRUD, activate/pause/cancel)
subscription	/api/v1/subscription-plans, /api/v1/subscriptions/me, /trial, /me/plan
bonification	GET /status, POST /transactions
webhook	/api/v1/admin/webhooks (CRUD, /rotate-secret, /test, /deliveries)
tenant	/api/v1/admin/api-keys, /api/kernel-core/actors/me
système/auth	/public/health, /api/v1/auth/login

2.6 Historique des migrations Flyway

La base est construite domaine par domaine, avec le wallet en premier socle, suivi de corrections tardives de cohérence schéma/entité — signe d'itérations rapides sur le mapping R2DBC :

Version	Contenu
V001-V007	Fondations wallet : table, transactions, demandes de paiement, logs d'audit, index, politiques, verrouillage optimiste
V008	Tables du moteur loyalty (règles, points, paliers, compteurs)
V009	Table tenants
V010	Table clés API
V011	Tables reward
V012	Tables referral
V013	Tables promo
V014	Tables campaign
V015	Tables subscription
V016	Ajout du mode de clé API
V017	Tables webhook
V018	Correction du schéma entité wallets (désalignement R2DBC)
V019	Élargissement de la colonne préfixe de clé API
V020	Conversion des enums wallet en varchar
V021	Cascade de suppression des livraisons webhook
V022	Correction du schéma entité wallet_policies

3 3. Frontend : portail Next.js

Le frontend est un portail d'administration Next.js 16 (App Router, TypeScript), organisé en routes internationalisées (src/app/[locale]/) avec next-intl (locales en par défaut et fr, résolues uniquement par la structure de dossiers, sans middleware.ts).

3.1 3.1 Portail admin (/portal)

Page	Rôle
/portal	Tableau de bord : agrège membres, règles actives et santé backend
/portal/rules	Liste et gestion des règles de fidélité (activation, création via modale)
/portal/establishment	Configuration de la politique de palier (tiers)
/portal/wallet/config	Politique du wallet (gel/dégel, plafonds)
/portal/members	Annuaire des membres avec recherche
/portal/members/[id]	Fiche membre détaillée (points, palier, wallet, historique, gel)
/portal/events	Bac à sable d'envoi manuel d'événements au moteur de règles
/portal/bonification	Statut de l'intégration externe + soumission de transactions
/portal/promo	CRUD des codes promo
/portal/campaigns	Campagnes temporisées (brouillon/active/pause/...)
/portal/subscriptions	Plans d'abonnement, abonnement courant, factures
/portal/logs	Journal tenant des transactions de points, paginé

Un **portail développeur séparé** existe sous /developer/* (clés API, webhooks, documentation, logs, sandbox), authentifié par clé API tenant plutôt que par JWT admin — deux mécanismes d'authentification cohabitent donc côté frontend :

- **JWT admin** (sessionStorage["loyalty_jwt_token"]) → en-tête Authorization: Bearer, pour /portal/* ;
- **Clé API tenant** (sessionStorage["loyalty_dev_api_key"]) → en-tête X-API-Key, pour /developer/*.

3.2 3.2 Couche d'accès API (src/lib/api.ts)

Point d'entrée unique (~700 lignes) vers le backend via le rewrite Next.js /backend/* → NEXT_PUBLIC_API_URL. Modules exportés : walletApi, memberApi, rulesApi, eventsApi, bonificationApi, systemApi, promoApi, campaignApi, subscriptionApi, authApi, apiKeyApi, webhookApi, devEventsApi, adminMembersApi, tierPolicyApi, adminLogsApi.

3.3 3.3 Hooks et composants

Un seul fichier src/hooks/useBackend.ts centralise l'ensemble des hooks de données, tous bâtis sur un hook générique interne (loading/error/refetch). Les composants sont séparés entre primitives shadcn/ui (components/ui/, à ne pas modifier directement) et composants de mise en page (components/layout/ : Header, Sidebar).

4 4. État d'avancement et feuille de route

D'après docs/TODO.MD et l'historique Git, le projet a suivi une feuille de route en 10 étapes : fondations → wallet → moteur de règles/points → bonification externe → récompenses → parrainage → codes promo → campagnes/paliers → abonnements SaaS → frontend. Les commits récents (« abonnement SaaS et frontend Dashboard », « frontend & connexion update », « connexion backend x frontend », « smoke test », « correction des pages affichant une erreur ») indiquent que le projet est actuellement en phase de **stabilisation de l'intégration frontend-backend** et de tests bout-en-bout (scripts scripts/smoketest_*.py), après une phase de construction backend largement complète.

5 5. Points forts et observations

- **Cohérence architecturale** : la même structure hexagonale et le même patron port/adapter se répètent sur onze domaines, avec vérification automatisée par ArchUnit — limite le risque de dérive.
- **Extensibilité du moteur de règles** : l'ajout d'un nouveau type de condition ou d'effet ne nécessite pas de modifier RuleEngine, grâce au patron Strategy.
- **Découplage propre des intégrations externes** : bonification et Kernel Core sont chacun isolés derrière un port dédié, avec adaptateur « désactivé »/repli pour ne pas bloquer le reste du système en cas d'indisponibilité.
- **Choix multi-tenant adapté au réactif** : propagation via Reactor Context plutôt que ThreadLocal, correct dans un contexte WebFlux.
- **Dette visible dans les migrations** : plusieurs migrations tardives (V018, V022) corrigent des désalignements entre entités R2DBC et schéma SQL — à surveiller lors de l'ajout de nouveaux champs.
- **Deux mécanismes d'authentification côté frontend** (JWT admin vs clé API tenant) qui cohabitent sans se recouper — cohérent avec la séparation portail admin / portail développeur, mais à documenter clairement pour de nouveaux contributeurs.